

Projectdocumentatie – AquaSense (Waterview)

Slim zwembadmonitorsysteem



Sven Van Leemput
Integratieproject - Toegepaste Informatica
AquaSense

Integratie van onderzoek, argumentatie en keuzes– AquaSense (Waterview)

Dit document verbindt de Business & Research-component met de uitvoering van Waterview / AquaSense. Per thema staat waarom een keuze logisch is gegeven probleemstelling, MVP-scope en doelgroep, niet alleen wat er gebouwd is.

1. Uitgangspunten uit het projectvoorstel

Het project vertrekt vanuit een praktisch probleem: zwembadwater wordt vaak nog handmatig gecontroleerd met teststrips. In de praktijk gebeurt dit niet altijd regelmatig, waardoor er geen duidelijk overzicht is van de waterkwaliteit doorheen de tijd. Dat kan invloed hebben op comfort, hygiëne en de installatie zelf.

Het doel van het project is daarom om metingen automatisch uit te voeren, centraal op te slaan en duidelijk weer te geven in een webdashboard.

De doelgroep bestaat vooral uit particuliere zwembadbezitters met beperkte technische kennis of weinig tijd. Daarom moest de interface eenvoudig en visueel begrijpelijk blijven.

De MVP focust op de belangrijkste functionaliteiten:

- temperatuur- en pH-metingen
- uploaden via WiFi
- een backend met API, databank en logging
- een dashboard met login, rollenbeheer en historiek/grafieken
- device-authenticatie via een sleutel of token

Extra functies zoals chloormetingen, realtime updates, notificaties en automatische dosering stonden in het projectvoorstel als nice-to-have. Deze zijn bewust buiten de eerste scope gehouden om het project haalbaar te houden.

De technische keuzes in de huidige codebase sluiten hierbij aan. Alles wat nodig is voor de must-haves is aanwezig, terwijl uitbreidingen later nog mogelijk blijven. Zo is er bijvoorbeeld al ondersteuning voorzien voor aparte threshold-configuratie, een optioneel chlorinePpm veld in het datamodel en een afzonderlijke dosing-interface zonder dat daar nu al hardware voor nodig is.

1.1 Doelgroepanalyse en persona's

De primaire doelgroep van AquaSense bestaat uit particuliere zwembadbezitters die hun waterkwaliteit willen opvolgen zonder dagelijks met teststrips bezig te zijn. Daarnaast is er een secundaire rol voor een beheerder of admin binnen demo- of kleine installaties.

Persona 1 - Tibo, particuliere zwembadbezitter

Tibo heeft een zwembad in de tuin, maar controleert de waterkwaliteit niet altijd regelmatig. Hij gebruikt af en toe teststrips, maar mist een duidelijk overzicht van de evolutie doorheen de tijd. Hij wil vooral snel kunnen zien of het water veilig en comfortabel blijft, zonder zich te verdiepen in technische details of chemie.

Persona 2 - Elise, druk gezin met zwembad

Elise onderhoudt het zwembad samen met haar partner. Metingen gebeuren meestal alleen bij de start van het seizoen of na een vakantie. Ze vergeet regelmatig controles uit te voeren en wil één eenvoudige plaats waar ze de actuele status, laatste metingen en trends kan bekijken. Een duidelijke interface met eenvoudige labels is voor haar belangrijk.

1.2 Empathy Map - Tibo

Onderdeel	Inhoud
Denkt	"Is mijn zwembadwater nog in orde?"
Voelt	Onzekerheid wanneer er lang niet gemeten werd
Ziet	Teststrips en losse waarden zonder historiek
Doet	Meet af en toe handmatig
Pains	Vergeten meten, weinig overzicht en technische onzekerheid
Gains	Automatische metingen, historiek en duidelijke status

1.3 Value Proposition

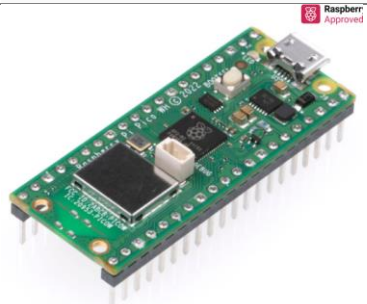
AquaSense helpt particuliere zwembadbezitters om hun waterkwaliteit eenvoudiger en regelmatig op te volgen. Automatische metingen worden gekoppeld aan een duidelijk dashboard met temperatuur, pH en historiek. Hierdoor krijgt de gebruiker sneller inzicht in afwijkingen en evoluties, zonder afhankelijk te zijn van losse teststrips of handmatige controles.

2. Microcontroller: Afweging en Keuze voor ESP32

Bij de keuze van de microcontroller werd gekeken naar verschillende criteria die belangrijk zijn binnen een IoT-project. Vooral WiFi-connectiviteit, prestaties, kostprijs en ondersteuning vanuit het ecosysteem speelden een grote rol.

Criterium	Relevantie voor dit project
Geïntegreerde WiFi	Nodig om metingen automatisch naar de cloud-API te sturen zonder extra hardware.
Rekenkracht en geheugen	HTTPS via WiFiClientSecure en JSON-verwerking vragen meer geheugen en verwerking dan een eenvoudige 8-bit microcontroller.
Beschikbaarheid en prijs	ESP32-modules zijn goedkoop en makkelijk verkrijgbaar voor prototypes.
Ecosysteem en libraries	Het Arduino-ecosysteem biedt veel bestaande libraries en voorbeelden voor sensoren, WiFi en HTTP-verkeer.

2.1 Vergelijking met alternatieven

	Alternatief	Waarom minder geschikt
	Arduino Uno + losse WiFi-module	Meer losse onderdelen, complexere opstelling en beperkte RAM voor HTTPS-verbindingen.
	ESP8266	Ondersteunt HTTP(S), maar heeft minder rekenkracht en geheugen dan de ESP32.
	Raspberry Pi Pico W	Mogelijk alternatief, maar minder voorbeelden en ondersteuning binnen het gebruikte project en ecosysteem.

De ESP32 bood uiteindelijk de beste balans tussen prestaties, eenvoud en ontwikkelsnelheid. Dankzij de ingebouwde WiFi, voldoende rekenkracht en brede ondersteuning binnen het Arduino-ecosysteem was dit een geschikte keuze voor het MVP van het project.

3. Sensoren: motivatie en beperkingen

3.1 Temperatuur - DS18B20

- De DS18B20 werd gekozen omdat deze sensor eenvoudig te integreren is via OneWire en veel gebruikt wordt in hobby- en IoT-projecten.
- De waterdichte uitvoering maakt hem geschikt voor metingen in zwembadwater of leidingen.



3.2 pH - analoge module + elektrode

- Voor de pH-meting werd gekozen voor een betaalbare analoge pH-module die rechtstreeks aangesloten kan worden op de ADC van de ESP32.
- De firmware gebruikt meerdere metingen, filtering, stabiliteitsdetectie en kalibratie-opslag om ruis en afwijkingen van goedkope elektroden op te vangen.
- Professionele sondes en automatische dosering werden bewust buiten de eerste scope gehouden.



3.3 Chloor

- Het datamodel laat chlorinePpm toe (nullable) zodat uitbreiding mogelijk blijft.

3.4 Fysieke prototype-opbouw (breadboard, verbindingen, randcomponenten)

Het projectvoorstel vermeldt naast microcontroller en sensoren ook breadboard, jumper wires en voeding.

Onderdeel	Rol in het prototype
Breadboard + jumper wires	Snelle aanpassingen en eenvoudige debugging tijdens ontwikkeling.
4,7 kΩ weerstand	Pull-up voor de OneWire-verbinding van de DS18B20.
LEDs (groen/geel/rood)	Lokale statusweergave van de waterkwaliteit via thresholds.
Drukknop	Wisselen tussen modi zonder firmware opnieuw te flashen.
Passieve buzzer	Audio-feedback bij waarschuwingen of moduswissels.
USB-voeding / adapter	Zorgt voor stabiele spanning voor sensoren en WiFi.

4. Systeemarchitectuur en modulariteit

Het voorstel beschrijft vier blokken:

- 1 IoT
- 2 Backend
- 3 Database
- 4 Webdashboard

Mijn implementatie volgt dat strikt:

- De applicatie gebruikt een duidelijke scheiding van verantwoordelijkheden. De ESP32 verwerkt de sensoren en kalibratie, terwijl de server instaat voor validatie en autorisatie. De gegevens worden opgeslagen in PostgreSQL en weergegeven via de Next.js-interface.
- Het prototype is modulair opgebouwd. Nieuwe parameters kunnen later toegevoegd worden via extra velden of tabellen zonder de bestaande werking te verbreken.
- Er werd bewust gekozen voor één centrale Next.js-applicatie. Daardoor blijven frontend en API in dezelfde codebase, met gedeelde types en validatie. Dit vereenvoudigt ontwikkeling en deployment.
- Een nadeel van deze aanpak is dat UI en API minder schaalbaar afzonderlijk zijn bij zeer hoge belasting.

5. Databank en data-model

PostgreSQL (via Supabase):

- Relatieel model voor devices ↔ profielen ↔ metingen
- Voor drempels met constraints op min/max paren.
- Tijdreeksen zijn querybaar met indexen op (deviceId, timestamp) - geschikt voor grafieken en paginatie zoals in het dashboard.

Afweging

- SQLite zou lokaal eenvoudiger zijn maar hindert hosted multi-user scenarios en gelijktijdige writes van web + devices.
- Een gespecialiseerde NoSQL- of time-series database zou beter schaalbaar zijn voor zeer grote hoeveelheden meetdata.

Daarom vormt PostgreSQL een goede middenweg tussen eenvoud, betrouwbaarheid en uitbreidbaarheid.

6. Authenticatie, rollen en ontwikkelsnelheid

6.1 Menselijke gebruikers

- Voor menselijke gebruikers maakt de applicatie gebruik van Supabase Auth. Dit biedt veilige sessies, login via e-mail en ondersteuning voor OAuth zonder dat er zelf wachtwoordbeheer, hashing of recovery-flows ontwikkeld moeten worden. Dat bespaart veel ontwikkelingstijd tegenover een volledig eigen authenticatiesysteem.
- App-laag: Profile koppelt Supabase userId aan rol (USER / ADMIN) en devices; eerste gebruiker wordt admin, praktisch voor demo en eerste installatie zoals in de handleiding.

6.2 Device authenticatie

- Voor devices wordt gewerkt met een API-key in de request header. In de databank wordt enkel een hash van deze sleutel opgeslagen, niet de plaintext-versie.
- Hierdoor blijven sleutels veiliger opgeslagen

Deze aanpak sluit aan bij het oorspronkelijke projectvoorstel waarin authenticatie via een API-sleutel of token voorzien was.

7. API-protocol: REST vs. WebSockets

Voor de MVP werd gekozen voor een REST API. Deze aanpak is eenvoudiger voor zowel de firmware als de backend en past beter bij periodieke metingen.

Aspect	REST (gekozen)	WebSockets / SSE
Firmware	Eenvoudige periodieke POST-requests	Permanente verbinding en reconnect-logica nodig
Backend	Stateless requests en eenvoudige route handlers	Extra connection management nodig
MVP-doel	Historiek en overzicht volstaan	Realtime updates zijn geen must

Conclusie: REST sluit direct aan bij MVP-scope en op IoT-kant. Updates kunnen later nog toegevoegd worden zonder de basisarchitectuur te wijzigen.

8. Beveiligingskeuzes

Bij dit prototype zijn enkele bewuste beveiligingskeuzes gemaakt. Ze beperken risico's bij communicatie, dataverwerking en beheer.

- TLS naar de cloud: de communicatie met de cloud verloopt via TLS. Dat is nodig omdat sleutels en metingen via het internet worden verstuurd. In het prototype is TLS voorzien.
- Geen vertrouwen in deviceld uit de body: metingen worden gekoppeld aan het device dat hoort bij de geldige sleutel. Ze hangen dus niet af van een vrij invulbaar deviceld-veld in de JSON. Dat verkleint de kans op een verkeerde koppeling door misconfiguratie of manipulatie.
- Inputvalidatie met Zod: elke externe payload wordt gecontroleerd met een vast schema. Zo is duidelijk welke data de API verwacht. Dit past binnen een contract-first aanpak in de developer stack.
- RBAC op de server: admin-routes controleren de rol van de gebruiker niet alleen in de interface, maar ook op de server. Dat is nodig omdat een publiek bereikbare API nooit enkel op de frontend mag vertrouwen.
- Auditlogging: belangrijke systeemactiviteiten worden gelogd. Dit maakt controle achteraf mogelijk en sluit aan bij het voorstel om systeemactiviteiten bij te houden.

9. Cachingstrategie

De prioriteit ligt bij betrouwbare en recente meetwaarden. Zwembadgebruikers hebben vooral actuele data nodig, zeker bij troubleshooting.

- Dashboardroutes gebruiken waar nodig force-dynamic, zodat Next.js actuele data uit de database ophaalt in plaats van statische HTML-cache te tonen.
- Middleware ververs Supabase-sessiecookies - correct gedrag voor ingelogde flows.
- Er is bewust geen aparte Redis-cache toegevoegd in de MVP. Dat houdt de setup eenvoudiger.
- Caching wordt pas interessant bij veel gebruikers of zware aggregaties. Nu gaat voorrang naar eenvoud en correctheid.

10. Gebruikerstests en feedback van testpersonen

10.1 Bevindingen en reacties

Thema	Wat er naar voren kwam
Eerste indruk dashboard	De eerste indruk was positief. Alle drie de testers vonden de statuskaarten duidelijk, vooral door de kleuren groen, oranje en rood. De recente grafiek gaf ook vertrouwen, omdat je meteen ziet dat er activiteit is.
Taal	De interface staat in het Engels. Twee testers vonden dat geen probleem. Eén tester gaf aan dat Nederlandse labels het systeem toegankelijker maken voor niet-IT-gebruikers.
Measurements / historiek	Het verschil tussen de korte grafiek op het dashboard en de volledige historiek was niet voor iedereen meteen duidelijk. Na uitleg via “Measurements” was dit helder. Een duidelijkere knop of link naar “alle metingen” blijft een UX-punt voor een latere iteratie.
Vertrouwen in data	Testers vergeleken de waarden reflexmatig met teststrips. Ze begrepen dat een prototype-sensor niet hetzelfde is als een labmeting. Een zichtbare tijdstempel of “last seen” helpt om te tonen hoe recent de meting is.
Dosing	De dosingfunctie werd nuttig gevonden als richtlijn. Wel moet duidelijk vermeld worden dat het om een schatting gaat en niet om automatische dosering via pompen.
Login / account	Inloggen met e-mail en wachtwoord verliep zonder problemen. Google-login werd tijdens de test niet gebruikt.

10.2 Verwerking in het ontwerp

- Verbeterpunten zoals Nederlandse labels en een duidelijkere link naar de historiek worden meegenomen na de MVP.
- Bij dosing blijft de nadruk liggen op een schatting.

11. Developer stack: afweging en keuze

Alternatief	Voordeel	Nadeel	Keuze
Next.js + TypeScript	Frontend en API in één project, type safety	Leercurve	Ja
Laravel	Sterke backendstructuur	Extra frontend nodig	Niet
Express + React	Flexibel	Meer losse configuratie	Niet
Nuxt/Vue	Goed alternatief	Minder passend bij gekozen stack	Niet